

Rubidium Programming Language Reference

File extension: `.rub`

Designed for beginners. Built to scale into real systems programming.

EXECUTION MODEL

Rubidium runs in two phases:

PHASE 1 – TOP LEVEL

Runs top to bottom before `main()`.

Creates variables in the global pool.

Executes any function calls written at the top level immediately.

Defines functions and classes but does NOT run them.

PHASE 2 – MAIN

After top level finishes, the compiler automatically runs `fn main()`.

```
print("This runs first – top level")

fn main() {
    print("This runs second – main")
}
```

MEMORY MODEL

All variables live in a SHARED GLOBAL POOL.

Rules:

- Variables are visible everywhere once created.
- There is no automatic memory management.
- Memory must be freed manually using `.drop()`.
- The only exceptions are `open()` file blocks and loop variables, which are freed automatically at the end of their block.

Think of memory as a shared whiteboard – everyone can see and modify it.

LANGUAGE PHILOSOPHY

Rubidium is designed around simplicity, predictability, and explicit behavior. Many modern languages use garbage collection, ownership systems, reference counting, borrow checking, or other hidden mechanisms. Rubidium instead favors rules that are visible and easy to reason about.

GLOBAL BY DEFAULT

Variables created normally are placed into the global memory pool.

```
let score = 100
```

Global variables can be accessed anywhere. Local variables must be declared explicitly.

```
let local temp = 100
```

Local variables never enter the global pool and are automatically dropped when their scope ends.

ASSIGNMENT ALWAYS COPIES

Assigning one value to another creates a complete deep copy.

```
let a = [1, 2, 3]
let b = a
```

a and b are completely independent.

Rubidium does not use references,
borrowing, or shared ownership.

MANUAL MEMORY MANAGEMENT

Memory is never freed automatically except
for special scoped values.

Normal variables remain alive until `.drop()`
is called.

```
let data = [1, 2, 3]
data.drop()
```

PREDICTABILITY OVER OPTIMIZATION

Rubidium favors behavior that is easy to
understand over behavior that is heavily
optimized.

The language intentionally prioritizes
clarity and consistency over maximum
performance.

NULL BEHAVIOR

Null is a valid value for every type.

```
let value: i32 = Null
```

Null is considered smaller than every
non-null value.

Conceptually, Null behaves like negative
infinity.

```
Null < 0           # True
Null < -999999     # True
Null == Null       # True
```

VARIABLES

Declare with 'let'. Type is optional – Rubidium infers it.

```
let x = 10
let name = "Rubidium"
let flag = True
```

Use 'mut' if you need to reassign the variable later.

```
let mut counter = 0
counter = 5
```

Trying to reassign a non-mut variable is a compile error.

```
let a = 10
a = 20 # ERROR: a is not mutable
```

DATA TYPES

Integers (signed):

i32, i64, i128, i256, i512, i1024, i2048

Floats:

f32, f64, f128, f256, f512, f1024, f2048

Other:

str – text string

bool – True or False

Null – valid for all types, means "exists but empty"

Any – can hold a value of any type

Collections:

list

index

dict

```
let a: i32 = 100
let b: f64 = 3.14
let c: str = "hello"
let d: bool = True
let e: i32 = Null
let f: Any = "I can be anything!"
```

Any can be used when you don't know the type in advance, for functions and return types and such

Strings are also indexable

```
let str: str = "Hello, World!"

for i in str.len() {
    print(str.char(i))
}
```

TYPE CASTING

Use 'as' to cast between types.

```
let y: i32 = 5
let large: i64 = y as i64
let f: f64 = 100 as f64
```

TYPE CHECKING

Rubidium checks types at compile time.

Assigning the wrong type raises a compiler error.

Example (will NOT compile):

```
let val: i32 = 5
```

```
val = "hello" # ERROR: wrong type
```

DEEP COPY RULE

Every assignment creates a full deep copy.

There is no borrowing or sharing of values.

```
let a = [1, 2, 3]
let b = a

b(0).set(999)

print(a)    # [1, 2, 3]  - a is unchanged
print(b)    # [999, 2, 3]
```

MATH

Standard operators:

+ addition

- subtraction

* multiplication

/ division

** power

*/ root

```
let a = 10 + 5      # 15
let b = 10 - 5      # 5
let c = 10 * 5      # 50
let d = 10 / 5      # 2
let e = 2 ** 3      # 8
let f = 2 */ 16     # 4
```

Standard BODMAS precedence applies.

```
let g = 1 + 2 * 3    # 7
let h = (1 + 2) * 3  # 9
```

Negate a value with a leading minus.

```
let x = 10
let y = -x           # -10
```

COMPARISON & LOGIC

Comparison operators:

`==` equal
`!=` not equal
`>` greater than
`<` less than
`>=` greater than or equal
`<=` less than or equal

```
let check: bool = 10 > 5    # True
```

Logical operators:

and

or

not

```
let result: bool = True and not False    # True
```

STRINGS

Strings store text.

```
let text = "Hello World"  
  
print(text)
```

Output:

Hello World

Strings use double quotes.

```
let name = "Max"  
let message = "Welcome to Rubidium"
```

Empty string

```
let empty = ""
```

Strings can contain numbers and symbols.

```
let text = "abc123!@#"
```

STRING METHODS

Get a character at an index.

```
text.char(index)

let text = "Hello"

print(text.char(0))
```

Output:

H

```
print(text.char(4))
```

Output:

o

Replace a character at an index.

```
text.set(index, value)

let mut text = "Hello"

text.set(0, "J")

print(text)
```

Output:

Jello

value must be a single character string.

```
text.set(4, "!")

print(text)
```

Output:

Hell!

Insert text before an index.

```
text.insert(index, value)

let mut text = "Hello"

text.insert(5, "!")

print(text)
```

Output:

Hello!

```
let text = "World"

text.insert(0, "Hello ")

print(text)
```

Output:

Hello World

Replace every matching character or string.

```
text.replace(old, new)

let mut text = "banana"

text.replace("a", "_")

print(text)
```

Output:

b_n_n_

```
let text = "Hello World"

text.replace("World", "Rubidium")

print(text)
```

Output:

Hello Rubidium

Convert a string into a list of characters.

```
text.slice()

let text = "Hello"

print(text.slice())
```

Output:

```
["H", "e", "l", "l", "o"]
```

```
let text = "Hello"

print(text.len())
```

Output:

5

COLLECTIONS

All three collection types (list, index, dict) require 'mut' if you want to modify them.

LIST

An ordered sequence. Zero-indexed.

```
let mut my_list: list = ["hello", 2.5, True]
```

Access by index:

```
print(my_list(0))  # hello
```

Mutate an item using .set():

```
my_list(0).set("changed")
```

Append a new item using .add():

```
my_list().add("new_item")
```

LIST UTILITIES

`.combine()` joins all values together.

```
let words = ["Hello", " ", "World"]

print(words.combine())
```

Output:

Hello World

INDEX

A flat key-value map.

Keys and values can be any type.

Each key stores exactly one value.

```
let mut my_index: index = [
    "name": "Rubidium",
    1: "one",
    2.5: "float key"
]
```

Access by key:

```
print(my_index("name"))    # Rubidium
```

Mutate a value using `.set()`:

```
my_index("name").set("updated")
```

Add a new key-value pair using `.add()`:

If the key already exists, this will error.

```
my_index().add("new_key", 99)
```

DICT

A key-value map where each key holds a collection of multiple values.

```
let mut my_dict: dict = {  
    "scores" = [10, 20, 30],  
    "names"  = ["alice", "bob"]  
}
```

Access a specific value inside a key:

dict(key, index)

```
print(my_dict("scores", 1))  # 20
```

Mutate a value using .set():

```
my_dict("scores", 1).set(999)
```

Add a value to an existing key's collection:

```
my_dict("scores").add(40)
```

Add a new top-level key:

The new key is created with value Null.

```
my_dict().add("new_key")
```

PRINT & INPUT

print() – outputs text and moves to a new line.

```
print("Hello, Rubidium!")  
print(x)  
print(i"Value is {x}")  
print("Count: " + x)
```

println() – overwrites the current console line.

Calling it again replaces the previous output.

```
println("Loading...")
println("Done!")
```

The console shows: Done!

input() – reads a line of text from the user.

```
let user: str = input("Enter your name: ")
```

FILES

Open a file using 'open()' as'.

The file automatically closes at the end of the block.

Any variables created inside the block persist,

but the file handle is gone after the closing brace.

```
open("data.txt") as file {

    file.write("Hello")      # overwrite entire file
    file.add(" World")      # add to end of file

    let data = file.read()   # read entire file
    print(data)

    let line = file.readLine(3) # read a specific line
    print(line)

    file.writeln(2, "Updated") # overwrite a specific line
}
```

Iterate over a file line by line:

```
open("data.txt") as file {

    for line in file {
        print(line)
    }
}
```

File utility functions (used outside of open blocks):

```
file.exists("data.txt")           # returns bool
file.new("data.txt")              # create a new file
file.delete("old.txt")            # delete a file
file.rename("old.txt", "new.txt") # rename a file
file.copy("save.txt", "backup.txt") # copy a file
```

File errors can be caught with try/error:

```
try {

    open("missing.txt") as file {
        print(file.read())
    }

} error {

    print("File error: " + error)
}
```

MEMORY — DROPPING VARIABLES

.drop() frees a variable's memory immediately.

The variable no longer exists after being dropped.

Collections recursively free all nested data.

```
let val = 42
val.drop()

let mut items: list = [1, 2, 3]
items.drop()
```

CONDITIONALS

```
let x = 7

if x == 7 {
    print("lucky seven")
} else if x > 10 {
    print("big number")
} else {
    print("something else")
}
```

LOOPS

While loop:

```
let mut count = 0

while count < 5 {
    print(count)
    count = count + 1
}
```

RANGE LOOP

`range(start, end)` – end is exclusive.

The loop variable is local to the loop only.

It can be named anything and is automatically dropped when the loop ends.

```
for i in range(1, 10) {

    if i == 5 {
        break
    }

    print(i)
}
```

The loop variable name is not special.

```
for value in range(1, 10) {  
    print(value)  
}  
  
for banana in range(1, 10) {  
    print(banana)  
}
```

COLLECTION ITERATION

Lists, indexes, dicts, and files can be used directly in for loops.

Loop variables are always local to the loop and are automatically dropped after execution.

```
let items: list = [10, 20, 30]  
  
for item in items {  
    print(item)  
}  
  
for number in items {  
    print(number)  
}
```

INDEX ITERATION

Iterating an index yields keys.

```
let table: index = [  
    "name": "Rubidium",  
    "lang": "Programming"  
]  
  
for key in table {  
    print(key)  
    print(table(key))  
}
```

DICT ITERATION

Iterating a dict yields keys.

```
let scores: dict = {  
    "players" = ["Alice", "Bob"],  
    "points" = [10, 20]  
}  
  
for key in scores {  
    print(key)  
    print(scores[key, 0])  
}
```

FILE ITERATION

Iterating a file yields lines.

The file handle remains valid only inside the open block scope.

```
open("data.txt") as file {  
  
    for line in file {  
        print(line)  
    }  
}
```

BREAK & CONTINUE

break exits the loop immediately.

```
for i in range(1, 10) {  
  
    if i == 5 {  
        break  
    }  
  
    print(i)  
}
```

continue skips the current iteration.

```
for i in range(1, 10) {  
  
    if i == 5 {  
        continue  
    }  
  
    print(i)  
}
```

FUNCTIONS

Functions share the global memory pool.

Function parameters are local to the function

and are automatically dropped when the function returns.

Imported files are isolated through namespaces.

Basic function:

```
fn greet() {  
    print("Hello!")  
}
```

With parameters:

```
fn add(a: i32, b: i32) -> i32 {  
    return a + b  
}  
  
let result = add(2, 3)    # 5
```

Return type is optional.

A function with no return type returns nothing.

```
fn log(msg: str) {  
    print(i"[LOG] {msg}")  
}
```

ERROR HANDLING

Use try/error to catch runtime errors.

The error variable holds the error message as a string.

```
try {  
  
    let bad: i32 = 10 / 0  
  
} error {  
  
    print("Error: " + error)  
}
```

try/error works with file operations too (see FILES section).

CLASSES

A class is a self-contained object template.

There is no 'self' keyword – class variables are accessed directly from inside the class.

```
class player() {  
  
    let mut health: i32 = 100  
    let mut name: str = "unnamed"  
    let mut scores: list = [0, 0, 0]  
  
    fn greet() {  
        print(i"Hi, I am {name}")  
    }  
  
    fn take_damage(amount: i32) {  
        health = health - amount  
    }  
  
    fn reset_scores() {  
        # Inside the class, collection fields are accessed directly  
        # without the instance name – just like non-collection fields.  
        scores(0).set(0)  
        scores(1).set(0)  
        scores(2).set(0)  
        scores().add(0)  
    }  
}
```

Create an instance:

```
let mut p = player()
```

Read class fields:

```
print(p.health)    # 100
```

Mutate a non-collection field using =:

The instance must be declared with mut.

```
p.health = 80  
p.name   = "Alice"
```

Mutate a collection field using .set():

From outside the class, prefix with the instance name.

```
p.scores(0).set(99)
```

From inside a class method, access directly without the instance name.

See reset_scores() in the class definition above.

Append to a collection field using .add():

```
p.scores().add(50)
```

Call a class method:

```
p.greet()  
p.take_damage(10)
```

Mutation rules:

- If the instance is immutable (no mut), nothing can be changed.
- If the instance is mutable (mut), only fields declared with 'mut' inside the class can be changed.

```
class locked() {  
    let value: i32 = 42    # can never be changed from outside  
}  
  
let mut obj = locked()  
obj.value = 99    # ERROR: value is not declared mut
```

CLASS MEMORY MODEL

Class instances do not store their fields

in the global memory pool.

Each instance owns its own data.

```
let mut a = player()  
let mut b = player()  
  
a.health = 50  
  
print(b.health) # 100
```

Assigning one instance to another creates

a complete deep copy.

```
let mut a = player()  
let mut b = a  
  
b.health = 10  
  
print(a.health) # 100  
print(b.health) # 10
```

THREADING

Rubidium supports real POSIX hardware threads.

```

use thread

fn task_one() {
    print("Task one done")
}

fn task_two() {
    print("Task two done")
}

fn run() {

    # thread(function(), id) – runs function asynchronously.
    # Thread IDs are integers, like timer IDs.
    # A thread ID can only be reused after that thread has finished.

    thread(task_one(), 1)
    thread(task_two(), 2)

    # thread.wait(id, ...) – blocks until all listed threads finish.

    thread.wait(1, 2)

    # thread.running(id) – returns True if the thread is still running.

    let still_going: bool = thread.running(1)
}

```

Threading creates a scope with special rules.

```

fn task_one(x) {
    let local y = 10
    print("done")
}

let z = 10

fn main() {
    thread(task_one(1), 1)
    thread(task_one(1), 2)
}

```

y is local, so each thread shares z but has its own x and y.

THREAD SCOPE RULES

Threads share the global memory pool.

Any global variable can be read or modified by any thread.

```
let counter = 0
```

All threads can access counter.

THREAD-LOCAL DATA

Function parameters and local variables belong only to the thread currently executing them.

```
fn worker(id) {  
    let local temp = 10  
  
    print(id)  
    print(temp)  
}
```

Every thread receives its own copy of:

- Function parameters
- Local variables
- Loop variables

These values are never shared between threads.

SHARED DATA

Global variables are shared.

```
let total = 0  
  
fn add() {  
    total = total + 1  
}
```

If multiple threads modify the same global

variable at the same time, the result is undefined.

Rubidium provides no automatic race condition protection.

AUTOMATIC CLEANUP

When a thread finishes:

- Function parameters are dropped
- Local variables are dropped
- Loop variables are dropped

Global variables remain alive.

THREAD MEMORY MODEL

Variable Type Shared?

Global Variable Yes

Function Parameter No

Local Variable No

Loop Variable No

File Handle No

Simple rule:

If it is in the global pool, it is shared.

Otherwise it belongs only to the currently executing thread.

RANDOM

use random

Generate a random number in a range.

random(min, max, type)

```
let r_int: i32 = random(0, 10, i32)
let rflt: f32 = random(0, 1, f32)
```

Shuffle a list in place:

```
let mut items: list = [1, 2, 3, 4, 5]
random.shuffle(items)
```

Pick a random value from a collection:

```
random.choice(items)

let mut my_index: index = ["a": 1, "b": 2]
random.choice(my_index)

let mut my_dict: dict = { "scores" = [10, 20, 30] }
random.choice(my_dict)
random.choice(my_dict("scores"))

random.seed("seed")
```

to set the starting seed

TIME

```
use time
```

Pause execution for a number of seconds.

```
time.wait(5)
```

Named timers – identified by integer IDs,

the same way thread IDs work.

Multiple timers can run simultaneously.

Start a timer.

Optionally specify a float return type for `timer_read`.

Defaults to `f32` if no type is given.

```
time.timer_start(1)          # returns f32 by default
time.timer_start(2, f64)     # returns f64
```

Pause a running timer (keeps elapsed time):

```
time.timer_pause(1)
```

Stop a timer (resets it):

```
time.timer_stop(1)
```

Read elapsed time in seconds.fractional seconds.

Returns the float type specified when the timer was started.

```
let elapsed = time.timer_read(1)
```

OS

```
use os
```

Start a hidden terminal session with an integer ID.

On Linux this opens a bash shell.

On Windows this opens cmd or powershell depending on your system.

The terminal is not visible – it is for Rubidium only.

```
os.start(1)
```

Run a command in a terminal session:

```
os.run(1, "ls -a")
```

Capture command output:

```
let output = os.run(1, "echo hello")
```

Pass input automatically when the command asks for it:

The third argument is sent as soon as input is requested.

```
let result = os.run(1, "sudo apt update", "y")
```

Drop a terminal session to free its resources:

```
os(1).drop()
```

FFI — FOREIGN FUNCTION INTERFACE

```
use FFI
```

Load a compiled shared library (.so on Linux, .dll on Windows).

The path starts from the folder your .rub file is in.

```
let c_lib = FFI("libs/mylib.so")
```

Bind a foreign function using `fn` with the FFI handle as the first identifier.

This does NOT define a new Rubidium function.

It creates a binding to the symbol inside the library.

```
fn c_lib my_c_func(msg: str) -> i32
```

Call the bound function like a normal function:

```
c_lib.my_c_func("hello from Rubidium")
```

Binding rules:

- The first identifier must be an FFI handle.
- The second identifier is the exact symbol name inside the library.
- The type signature is used for ABI mapping and compile-time type checking.
- Bindings are resolved at load time.
- Compatible with C, C++, C#, and Rust shared libraries.

MODULES & IMPORTS

'use' enables built-in compiler-provided modules.

A module that is not declared with 'use' cannot be accessed.

```
use thread
use random
use time
use os
use FFI
```

'import' loads an external .rub file from the same folder into your code with the name of the file as the namespace

```
import math_tools

math_tools.sin(0.5)
```

MAIN ENTRY POINT

The compiler always looks for `fn main()` as the program entry point.

It runs after all top-level code has finished.

```
fn main() {
    print("Welcome to Rubidium!")
}
```

NOTES

Syntax:

- Statements end with a newline
- Semicolons are not required
- Blocks use braces { }
- Comments use
- Strings use double quotes
- String interpolation uses: `i"text {variable}"`
- Boolean values are `True` and `False`
- Logical operators are: `and`, `or`, `not`
- Type casting uses: `as`
- Square root uses: `*/value`

Memory:

- All assignments create a full deep copy
- Values are never borrowed or shared
- Variables live in the global pool by default
- Memory must be freed manually with `.drop()`
- Accessing dropped memory causes a runtime error
- Loop variables are automatically dropped when the loop ends
- File handles are automatically dropped when an `open()` block ends
- Null is a valid value for all types (exists but empty)
- `null == null` is true

Scope:

- Variables created at the top level are visible everywhere
- Functions can access global variables
- Function parameters exist only for the lifetime of the function
- Classes create their own scope
- Imported files create their own namespace

Types:

- Type checking happens at compile time
- Null is valid for every type
- Any can store any value type
- Assigning an incompatible type causes a compile error
- Unknown or missing symbols raise compile errors (error if fn/class not found)
- Reassignment rules depend on `mut` (non-mut = compile error)
- Not all errors are runtime; many are compile-time by design

Null:

- Null represents an existing value with no data
- Null can be compared using `==` and `!=`
- Null is considered smaller than all non-null values
- Example: `Null < 0` is True
- Null comparisons are valid and well-defined across types

Collections:

- Collections require `mut` to be modified

- list = ordered sequence of values
- index = key -> value map
- dict = key -> collection map
- .len() returns the number of items
- .has() checks for keys or values
- Accessing a missing key or index causes a runtime error
- Iteration behavior:
 - * list -> values
 - * index -> keys
 - * dict -> keys
 - * file -> lines
- Collections use deep copy on assignment
- you can force types like let x: list: i32 = [0, 10, 32]

Collection Equality:

- Collection equality compares contents
- Two collections with identical contents are equal
- Equality is checked recursively
- null values inside collections are valid and comparable

Strings:

- Strings are zero indexed
- .len() returns number of characters
- .char(index) returns character at index
- .set(index, value) replaces a character
- .set() requires a single-character string
- .insert(index, value) inserts text before index
- .replace(old, new) replaces all matches
- .slice() returns list of characters
- String modification requires mut
- Strings are iterable in loops

Loops:

- Loop variable names can be anything

- Loop variables are local to the loop
- Loop variables are automatically dropped after the loop
- Lists yield values
- Indexes yield keys
- Dicts yield keys
- Files yield lines
- User-defined classes cannot define custom iteration behavior
- Loop variables are not persistent outside loop scope

Range:

- `range(start, end)` excludes the end value
- If `start < end`, counts upward
- If `start > end`, counts downward

Indexes:

- Iteration preserves insertion order
- Keys can be any type (not just strings)

Classes:

- Class instances follow normal deep-copy rules
- Assigning one instance to another creates a full copy
- Class fields marked `mut` may be modified
- Non-`mut` fields cannot be modified
- No `"self"` keyword; fields are accessed directly
- Methods operate on instance state implicitly
- Attempting to modify locked fields causes compile error

Threads:

- Threads share access to global variables
- Multiple threads may modify the same variable
- No race-condition protection is provided
- Invalid memory access causes runtime error
- Thread IDs are required for control and synchronization

Imports:

- `use` enables built-in modules

- `import` loads external `.rub` files
- Imported files are accessed through their namespace
- Built-in modules and imported namespaces are separate systems
- Missing imports or symbols result in compile errors

Files & Memory Notes:

- `open()` blocks auto-close file handles
- File variables persist after block ends
- File operations support try/error handling
- File iteration yields lines only inside open scope

Errors (new clarified behavior):

- Some errors are compile-time (missing fn/class, bad types)
- Some are runtime (invalid memory access, invalid indexing)
- mut violations cause compile errors
- Accessing null behaves as valid but empty state

Local Variables:

- Declared using `'local'`
- Syntax: `let mut local x: type = value`
- Exist only inside the current block or function
- Automatically dropped at scope exit
- Cannot be accessed outside their scope
- May shadow global variables
- Do not enter the global pool
- Follow normal mutability rules